RFID Tray Tracking

Overview
The RFID Tray Tracking system is designed to identify trays and determine their corresponding lot numbers using RFID technology. Each tray is associated with an RFID tag, allowing quick and accurate identification through scanning.

The solution has been implemented using Ignition Perspective and can be further integrated into **Kanoa MES** for production-level deployment.

Objective
To scan RFID-tagged trays and automatically retrieve the associated lot number, ensuring accurate tracking and reducing manual input errors.

Components Required

1. **RFID Reader/Writer (Yanzeo)** – Used to read RFID tag data and transmit it to the system
2. **RFID Sticker Tags** – Attached to trays, containing unique identification numbers
3. **Ignition Designer Software** – Used to develop the user interface and logic for scanning and validation

Working Principle

- Each tray is assigned a unique RFID tag (tray number).
- When scanned, the RFID reader sends the tag value as input to Ignition.
- The system checks the scanned RFID against a predefined mapping of tray numbers and lot numbers.
- If a match is found, the corresponding lot number is displayed.
- If no match is found or a mismatch occurs, then it says not Lot has been found.

YANZEO SR360 UHF RFID Reader Setup

Device Description

The Yanzeo SR360 (865–915 MHz) Desktop UHF RFID Reader/Writer is used for reading and writing RFID tags. It supports keyboard emulation (USB HID mode), allowing scanned tag data to be directly input into applications such as Ignition Perspective.

Important Note
Since the reader operates within the 865–915 MHz UHF frequency range, it is essential to use UHF RFID tags.
Using HF or LF tags will not work with this reader.

Figure 1: Yanzeo RFID Reader/Writer

Package Contents

- Yanzeo SR360 RFID Reader
- USB Cable (for power and data communication)
- Sample UHF RFID Tag

Software Requirement
The reader requires configuration software for writing and managing RFID tags.

Download the software from the official Yanzeo website(https://www.yanzeo.com/rfid-write-reader/handheld-rfid-readers-and-rfid-enabled-scanners.html)

Go to Yanzeo Official Website>Search button>type SR360>Enter >Scroll Down> Go to Manual SDK Attachments(where u can find all the documents related to the Reader and

also the software required to configure the reader and writing the tags. (**Figure Below, shows the steps:)**

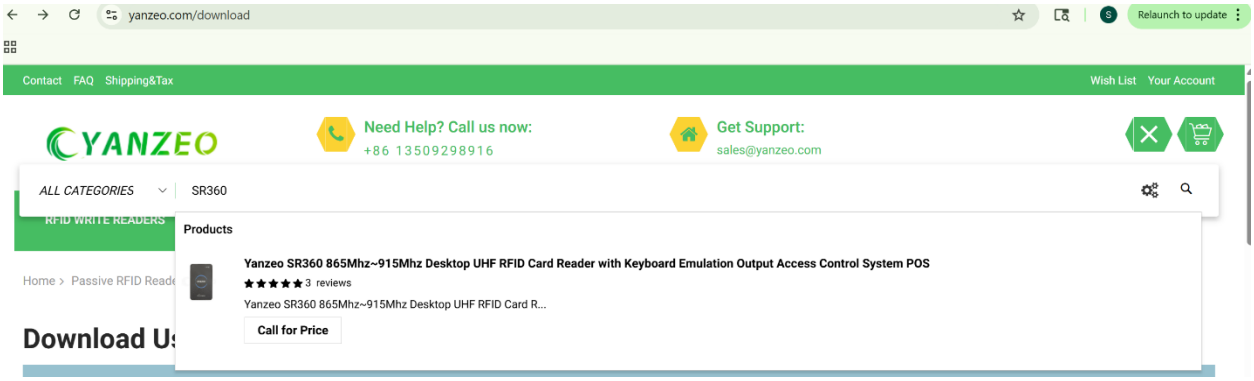


Figure 2: Step 1

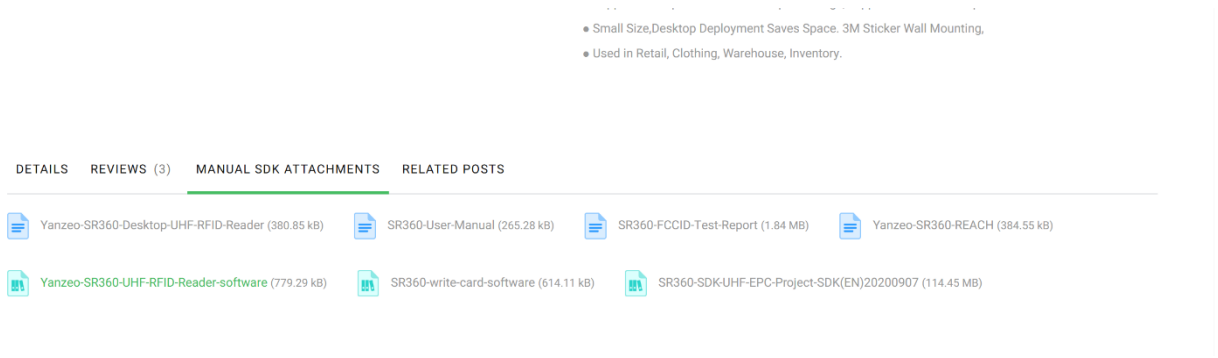


Figure 3: Download the Software

Open the Software:

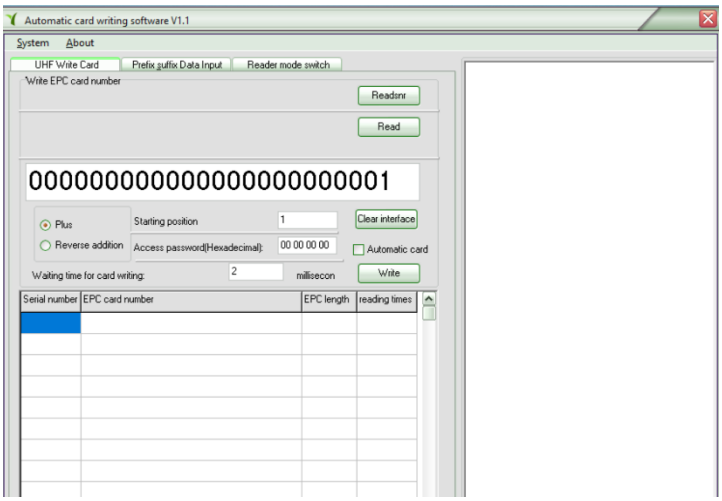


Figure 4: Software Interface

Since the reader is the keyboard emulator, so for reading the tags using the software , got to top bar Reader Mode Switch(red Circled) then switch back to UHF Write Card window and

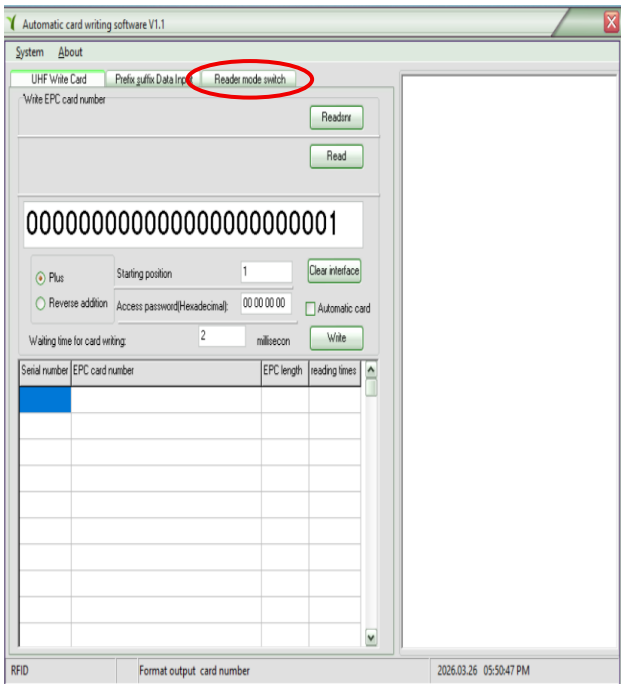


Figure 6: Reader Mode switch

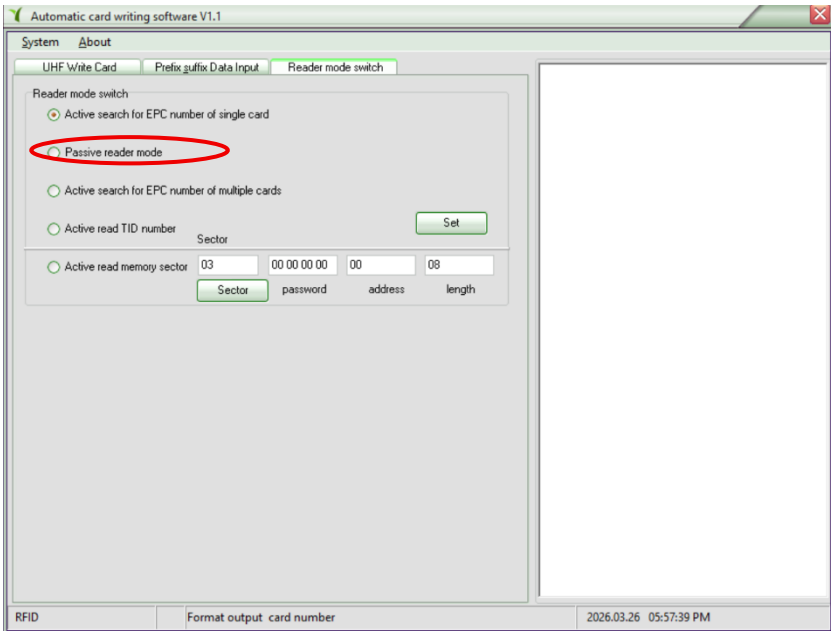


Figure 5: Reader Passive Mode Selection

Placing the card on top of the reader , press Read on the Software and it will show the value of the card in the white blank screen.

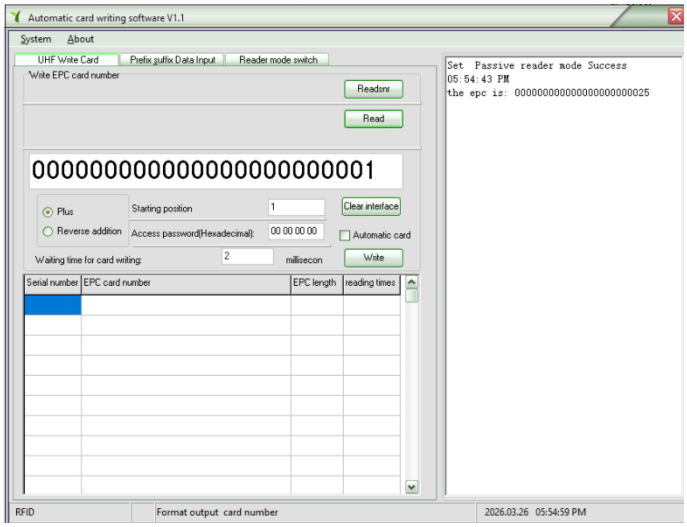
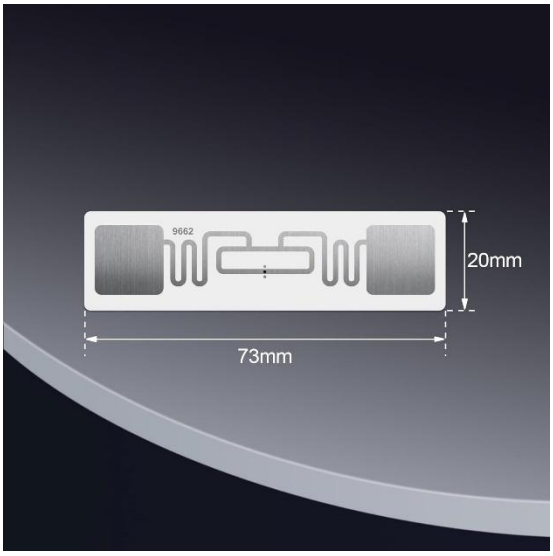


Figure 7: RFID Tag Value

Note : After reading it Open Notepad/Word/Excel to verify the tag number and before closing the software its better to turn off the passive mode and turn on the Active Search for EPC number or single card.

Writing Tag using Software:

- **UHF RFID Tag Sticker, Rewritable Adhesive Inlay Label,AZ9662 73x20mml SO18000-6C Long Range EPC Class1 Gen2 for Warehouse Management (Frequency: Ultra High Frequency 860-960mhz Compatible)**



Link to buy: [Amazon.com: UHF RFID Tag Sticker, Rewritable Adhesive Inlay Label,AZ9662 73x20mml SO18000-6C Long Range EPC Class1 Gen2 for Warehouse Management（Pack of 100 : Industrial & Scientific]

Follow the same procedure as “reading” the tags make the reader to Passive Mode.

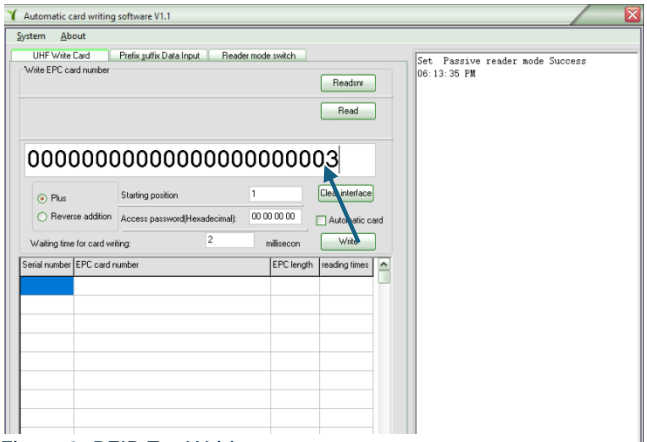


Figure 8: RFID Tag Writing

Then Come back to UHF Write Card window. Put the desired ID number on the box (Figure 8) , and press the “Write” right below it.

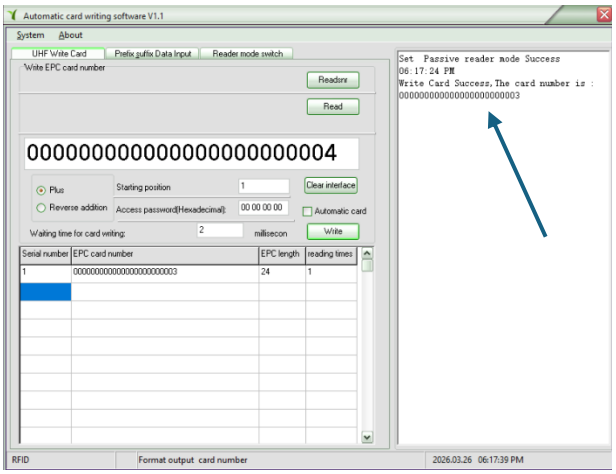


Figure 9: Writing Successful

It will write the card successfully. On the writing screen, it will automatically show the next number. For example, if the card written has number 3, the next number displayed will be 4.

Note: Sometimes the card may not write properly. In that case, unplug the reader and plug it back in. Then go to Reader Mode Switch, set it to **Passive Mode**, and follow the same steps again. The card should be written successfully.

Ignition Perspective

Overview:

The goal is simple: when an operator scans an RFID tag using a barcode scanner, the system looks up the corresponding Lot Number and displays it in a popup. If the tag is not recognized, a 'Not Found' message is shown instead.

Component Used — Barcode Scanner Input

The Barcode Scanner Input component in Ignition Perspective was used to capture the scanned RFID tag value. This component acts like a keyboard input — when a scan happens, it captures the keystrokes and stores the result internally.

Event Used: onActionPerformed — fires every time a scan completes.

The component stores scanned values in a list called data inside its props. Each new scan appends to this list:

self.props['data'] → ['24', '25', '24', '26']

To always get the most recently scanned value, we use index [-1] which reads the last item in the list:

```
scanned_tray = str(self.props['data'][-1]).strip()
```

Regex Configuration on the Component

The Barcode Scanner Input component has a regex property that filters and cleans the raw input from the scanner. Scanners often append invisible characters like newlines or carriage returns at the end of a scan.

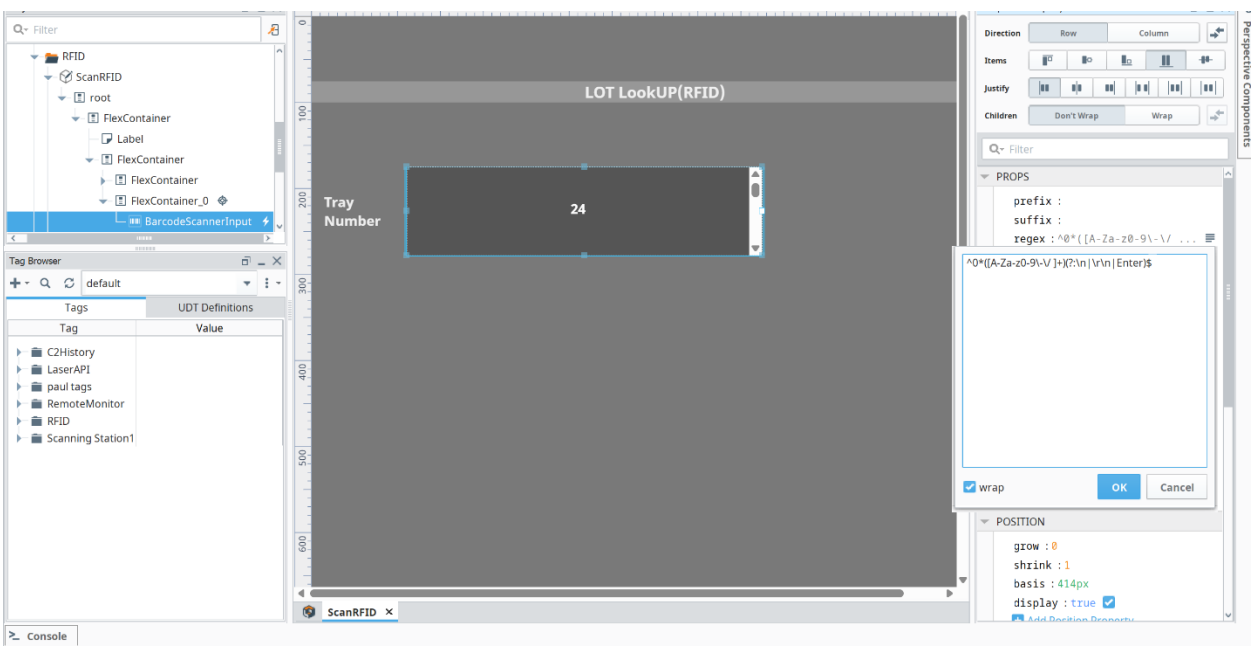
The regex configured on the component was:

```
^0*([A-Za-z0-9\-\/ ]+)(?:\n|\r\n|Enter)$
```

What this does:

- Strips leading zeros from the scanned value
- Accepts alphanumeric characters, dashes, slashes, and spaces
- Strips the trailing newline or Enter character that scanners typically send

This ensures the scanned value is clean before the script processes it.



How the Code Works

The full script runs on onActionPerformed. Here is a breakdown of each part:

Step 1 — Read the Scanned Value

```
scanned_tray = str(self.props['data'][-1]).strip()
```

Gets the last scanned value from the component's data list and converts it to a clean string.

Step 2 — Dictionary Lookup

```
tray_lot_map = {
    "24": "2600-002",
    "25": "2600-003",
    "26": "2600-004"
}
```

Instead of using memory tags or hardcoded if/elif chains, a Python dictionary maps each tray number to its corresponding lot number. This makes it easy to add new trays without changing any logic.

Step 3 — Open Popup with Result

```
if scanned_tray in tray_lot_map:
    lot_number = tray_lot_map[scanned_tray]
    message = lot_number
else:
    message = "Lot Not Found: Tray " + scanned_tray

system.perspective.closePopup('View2PopUp')
system.perspective.openPopup(
    'View2PopUp', 'RFID/View2',
```

```
    params={"passedLotNumber": message},
    modal=True, overlayDismiss=True,
    position={'width': 600, 'height': 200}
)
```

The popup view RFID/View2 receives the lot number (or not found message) as a parameter called passedLotNumber. Inside the popup, the label is bound to view.params.passedLotNumber to display it.

Step 4 — Clear Old Scan Data

```
finally:
    self.props['data'] = [self.props['data'][-1]]
```

After every scan, the data list is trimmed to keep only the last scanned value. This prevents the list from growing indefinitely while still keeping the last result visible until the next scan.

5. Popup Behavior

The popup opens centered on the screen using modal=True which automatically handles centering. Key behaviors:

- modal=True centers the popup automatically — no manual position coordinates needed
- overlayDismiss=True allows the operator to click outside to close
- closePopup is called before openPopup so scanning a new tag instantly replaces the old popup without needing to close it manually

Note: Several issues were encountered during development before the final working solution was reached:

Problem	What Was Assumed	What Actually Happened
Memory tag always returned 24	Tray Number tag would update on each scan	Memory tag was hardcoded to 24 and never changed regardless of what was scanned
self.props.value failed	props was an object accessible with dot notation	props is a dictionary — correct key is props['data'], not props.value
event was empty	event would contain the scanned value like event.newValue	onActionPerformed passes an empty dictionary — scanned value lives in the component props
else block never ran	The if/else logic was broken	scanned_tray was always '24' due to the memory tag issue, so else never had a chance to run
openPopup returned None	openPopup returned a component reference to modify	openPopup just opens the popup and returns nothing — data must be passed as params
Old popup stayed open	Scanning a new tag would refresh the popup	Ignition does not auto-refresh an open popup — closePopup must be called first

Final Script:

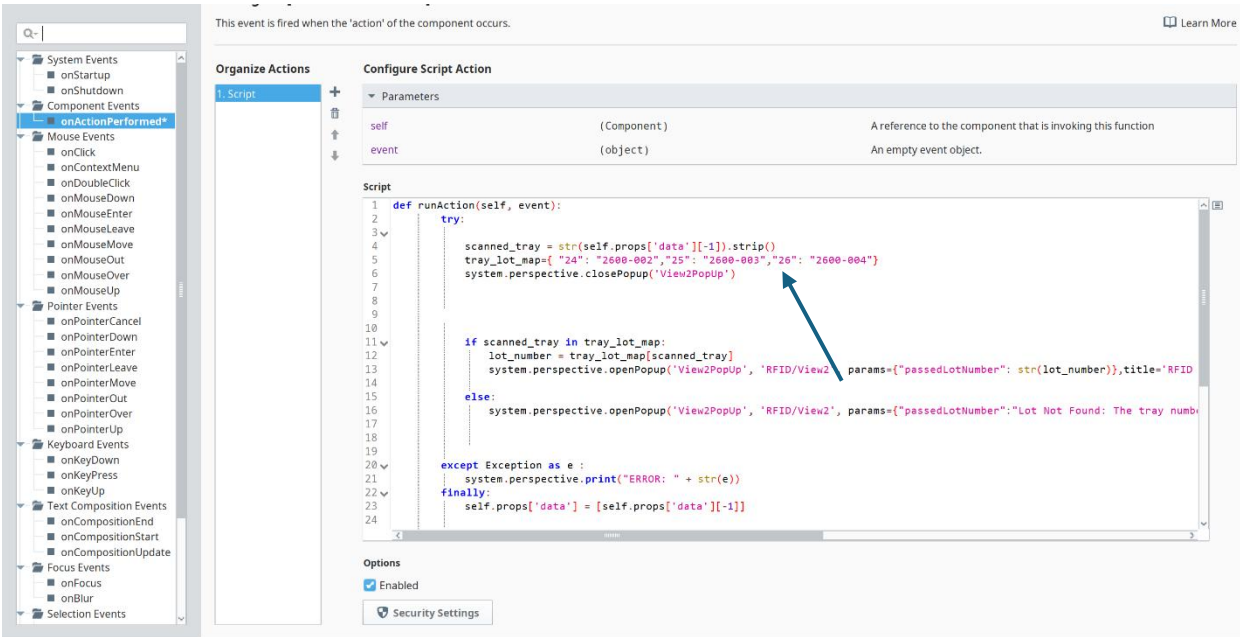
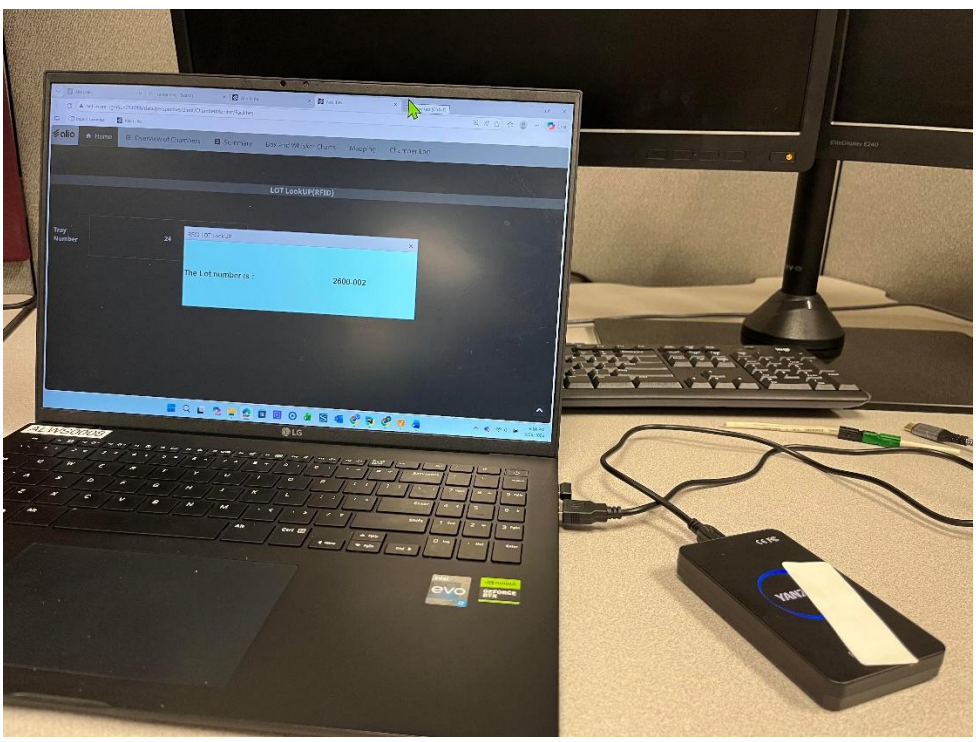


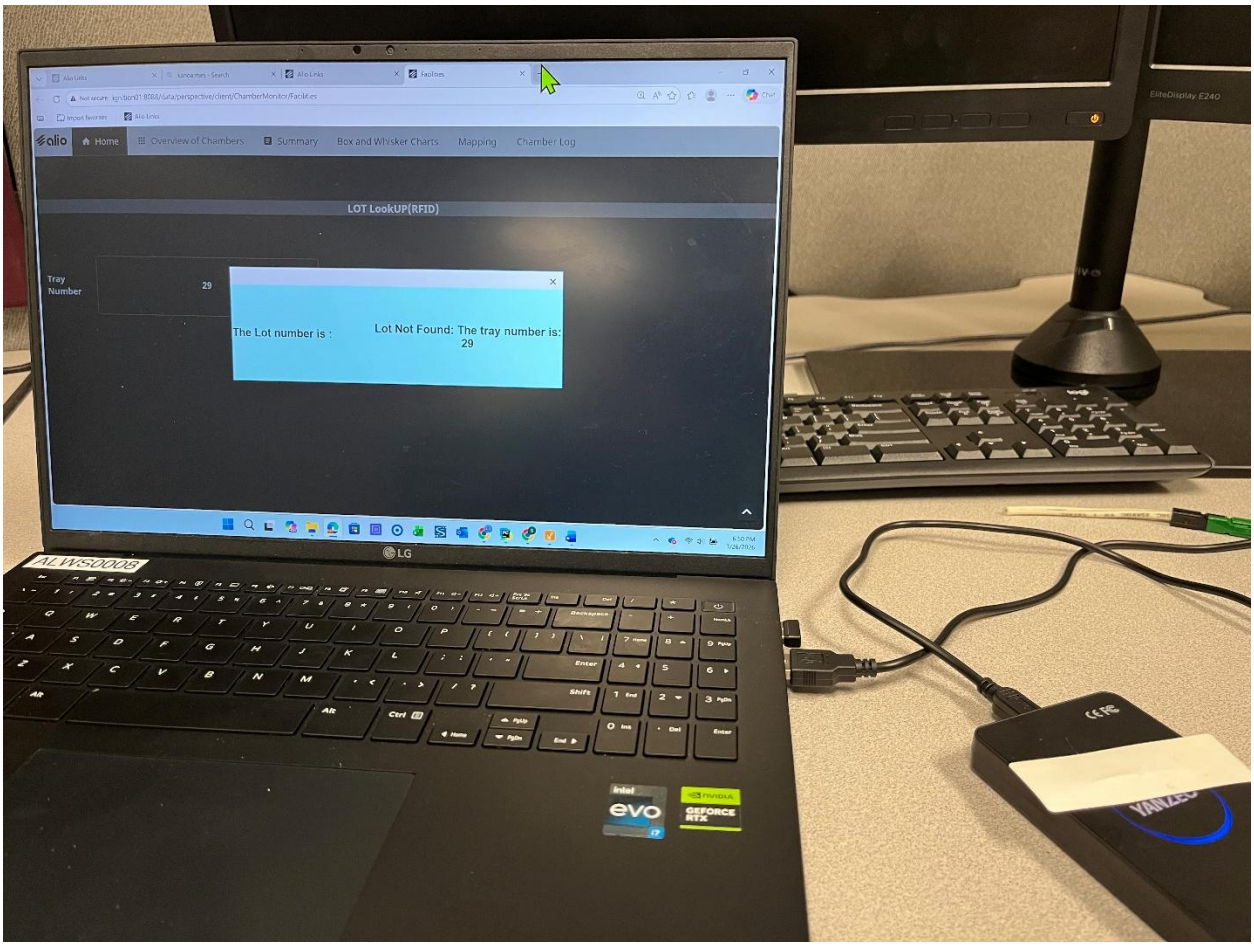
Figure 10: Script

Test 1:



The tray number is in the dictionary which has been hardcoded to the code (Arrow on Figure 10)

Test 2:



Here, there are no trays whose number is 29 in the hardcoded Dictionary. So it opens a pop up and says no lot number found and the tray number is 29